11.4 Pointer Safety

At the beginning of the previous chapter, we said that the hard thing about pointers is not so much manipulating them as ensuring that the memory they point to is valid. When a pointer doesn't point where you think it does, if you inadvertently access or modify the memory it points to, you can damage other parts of your program, or (in some cases) other programs or the operating system itself!

When we use pointers to simple variables, as in section 10.1, there's not much that can go wrong. When we use pointers into arrays, as in section 10.2, and begin moving the pointers around, we have to be more careful, to ensure that the roving pointers always stay within the bounds of the array(s). When we begin passing pointers to functions, and especially when we begin returning them from functions (as in the `strstr` function of section 10.4) we have to be more careful still, because the code using the pointer may be far removed from the code which owns or allocated the memory.

One particular problem concerns functions that return pointers. Where is the memory to which the returned pointer points? Is it still around by the time the function returns? The `strstr` function returns either a null pointer (which points definitively nowhere, and which the caller presumably checks for) or it returns a pointer which points into the input string, which the caller supplied, which is pretty safe. One thing a function must *not* do, however, is return a pointer to one of its own, local, automatic-duration arrays. Remember that automatic-duration variables (which includes all non-static local variables), including automatic-duration arrays, are deallocated and disappear when the function returns. If a function returns a pointer to a local array, that pointer will be invalid by the time the caller tries to use it.

Finally, when we're doing dynamic memory allocation with `malloc`, `realloc`, and `free`, we have to be most careful of all. Dynamic allocation gives us a lot more flexibility in how our programs use memory, although with that flexibility comes the responsibility that we manage dynamically allocated memory carefully. The possibilities for misdirected pointers and associated mayhem are greatest in programs that make heavy use of dynamic memory allocation. You can reduce these possibilities by designing your program in such a way that it's easy to ensure that pointers are used correctly and that memory is always allocated and deallocated correctly. (If, on the other hand, your program is designed in such a way that meeting these guarantees is a tedious nuisance, sooner or later you'll forget or neglect to, and maintenance will be a nightmare.)

Read sequentially: prev next up top